

QA Automation Engineering Case Study

Candidate: Rohan Patil | Applied Role: Automation Engineering Intern

Part 1: Debugging Flaky Test Code

1. Identifying Flakiness Issues & 2. Root Causes

The provided Playwright test code will likely experience intermittent failures (flakiness) in a CI/CD pipeline due to the following reasons:

- **Missing Explicit Waits for Dynamic Content:** The original code calls `page.click("#login-btn")` and immediately asserts the URL and visibility of the dashboard elements. Since the application uses dynamic loading, the DOM takes time to render, and network requests take time to resolve. CI/CD runners usually have different network latency than local environments, causing the test to run faster than the UI updates.
- **Premature Element Fetching:** In `test_multi_tenant_access`, `page.locator(".project-card").all()` is executed instantly after the login click. If the dashboard hasn't loaded, this returns an empty list, completely bypassing the assertions without failing, or failing if elements attach mid-execution.
- **No 2FA Handling:** The requirements mention that 2FA is present for some users. The current code assumes a direct transition from login to dashboard, which will fail if an intermediate 2FA screen appears.

3. Provided Fixes

Below is the refactored code. I have implemented Playwright's auto-retrying assertions (`expect`) and explicit waits to ensure stability across different environments.

```
import pytest
from playwright.sync_api import sync_playwright, expect
import os

def test_user_login():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        page.goto("https://app.workflowpro.com/login")

        # Ideally, credentials should be fetched from environment variables, not hardcoded
        page.fill("#email", "admin@company1.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        # Fix 1: Wait for navigation to complete before asserting.
        # This handles variable tenant loading times in CI.
```

```

page.wait_for_url("**/dashboard", timeout=10000)

# Fix 2: Use auto-retrying expect to handle dynamic DOM rendering
welcome_message = page.locator(".welcome-message")
expect(welcome_message).to_be_visible(timeout=10000)

browser.close()

def test_multi_tenant_access():
    with sync_playwright() as p:
        browser = p.chromium.launch()
        page = browser.new_page()

        page.goto("https://app.workflowpro.com/login")
        page.fill("#email", "user@company2.com")
        page.fill("#password", "password123")
        page.click("#login-btn")

        page.wait_for_url("**/dashboard")

        # Fix 3: Ensure the element is actually attached to the DOM before fetching all
        page.wait_for_selector(".project-card", state="visible", timeout=15000)

        projects = page.locator(".project-card").all()
        assert len(projects) > 0, "No projects loaded on the dashboard"

        for project in projects:
            assert "Company2" in project.text_content()

        browser.close()

```

Part 2: Test Framework Design

To support a multi-tenant B2B SaaS platform across web and mobile, I propose a hybrid Data-Driven and Page Object Model (POM) framework using Python, pytest, and Playwright.

1. Framework Structure

```

workflowpro-automation/
├─ api_clients/           # API helper classes (e.g., project_api.py, auth_api.py)
├─ page_objects/         # Web elements & actions (e.g., login_page.py,
dashboard_page.py)
├─ tests/
│   ├─ api/              # Pure API tests
│   ├─ ui/               # Web UI tests
│   └─ integration/      # End-to-end flows combining API + UI
├─ test_data/            # JSON files for tenant configs, user roles
└─ utils/                # BrowserStack config, custom logger, DB teardowns

```

```
└─ conftest.py          # Pytest fixtures (setup/teardown, auth state, driver init)
└─ pytest.ini           # Pytest configurations and markers
└─ requirements.txt
```

2. Configuration Management

- **Environments & Tenants:** Use a `.env` file or JSON config files mapped to pytest CLI arguments (e.g., `pytest --env=staging --tenant=company1`). Based on the argument, the framework builds the correct base URL dynamically.
- **Browsers & Cross-Platform:** Playwright natively supports cross-browser testing via CLI flags (`--browser=firefox`). For mobile, I would utilize Playwright's mobile emulation devices in `conftest.py`, or route the tests to BrowserStack using a custom fixture that initializes a Remote Web Driver based on the desired OS/Device capabilities.

3. Missing Requirements & Assumptions (Questions to ask)

- **Test Data Management:** How do we handle test data isolation? Should tests seed their own data via API and clean it up (teardown), or is there a static staging database?
- **Reporting:** What reporting tool is preferred for CI integration? (I would recommend Allure for comprehensive API/UI reports).
- **2FA Automation:** For the admin roles, how is 2FA handled during automation? Are we using static bypass codes, or integrating with an authenticator API?

Part 3: API + UI Integration Test

This test utilizes a hybrid approach: creating data via API for speed, and verifying via UI for user confidence. Having worked with robust data pipelines and REST APIs (like TMDb) in Python, I prefer isolating API interactions into reusable client helpers to keep the test logic clean.

Integration Test Implementation

```
import pytest
import requests
from playwright.sync_api import expect

# ASSUMPTION: API endpoints and tokens are managed via pytest fixtures
def test_project_creation_flow(api_context, web_page, mobile_page, auth_token):

    # 1. API: Create project
    headers = {
        "Authorization": f"Bearer {auth_token}",
        "X-Tenant-ID": "company1_id"
```

```

    }
    payload = {"name": "E2E Automation Project", "description": "Testing Integration"}

    response = requests.post("https://api.workflowpro.com/api/v1/projects", json=payload,
headers=headers)
    assert response.status_code == 201
    project_id = response.json()["id"]

    # 2. Web UI: Verify project display on desktop browser
    web_page.goto("https://company1.workflowpro.com/projects")
    project_locator = web_page.locator(f"[data-project-id='{project_id}']")
    # Expect handles implicit waiting for network completion
    expect(project_locator).to_be_visible(timeout=10000)
    expect(project_locator).to_contain_text("E2E Automation Project")

    # 3. Mobile: Check mobile accessibility
    # (mobile_page is a fixture utilizing Playwright's mobile emulation or BrowserStack)
    mobile_page.goto("https://company1.workflowpro.com/projects")

    # Handle responsive edge cases: e.g., opening a hamburger menu on mobile
    menu_button = mobile_page.locator(".mobile-menu-toggle")
    if menu_button.is_visible():
        menu_button.click()

    mobile_project = mobile_page.locator(f"[data-project-id='{project_id}']")
    expect(mobile_project).to_be_visible()

    # 4. Security: Verify tenant isolation (Negative Testing)
    # Switch tenant API header to Company 2 and verify project access is forbidden
    headers_tenant2 = {
        "Authorization": f"Bearer {auth_token}",
        "X-Tenant-ID": "company2_id"
    }
    isolation_response = requests.get(f"https://api.workflowpro.com/api/v1/projects/{project_id}", headers=headers_tenant2)
    assert isolation_response.status_code in [403, 404], "Tenant isolation failure!"

    # Teardown logic would typically exist in a pytest fixture yield block
    # to cleanly delete the project via API after the test completes.

```

Testing Strategy & Edge Cases Considered

- **API First:** Using API for data setup is much faster and less flaky than driving the UI to create a project before testing the display.
- **State Management:** In a real implementation, authentication state (cookies/tokens) would be saved and reused via Playwright's `browser_context.storage_state` to skip the UI login step entirely, speeding up execution.

- **Responsive Design:** The test checks if mobile-specific elements (like a hamburger menu) are visible before interacting, accommodating CSS media query changes.